

Hệ thống nhúng payload mang bằng chứng ZK vào video H.264/CAVLC

Tài liệu kỹ thuật dựa trên mã nguồn và artifact benchmark của dự án

25 tháng 9 năm 2026

1 Kiến trúc hệ thống

Hệ thống có ba phần chạy ở các thời điểm khác nhau: ứng dụng tạo proof cho một message; pipeline video chọn và sửa các bit dấu của coefficient CAVLC; phía nhận khôi phục message/proof rồi verifier kiểm tra proof bằng public signals và verification key. Camera không tạo proof cho từng frame: proof được tạo trước, còn video là kênh mang payload.

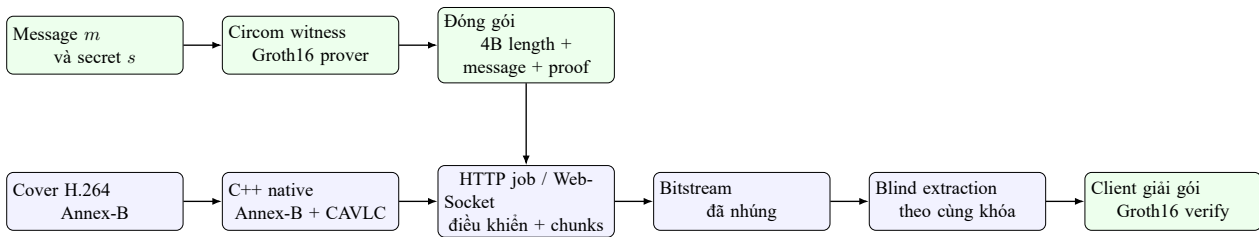


Figure 1: Các khối ứng dụng và video. Dữ liệu được nhúng trong residual H.264; hệ thống không dùng SEI làm nơi chứa payload.

Hình 1 phân biệt proof với carrier: Groth16 tạo bằng chứng một lần cho message, còn C++ chỉ xử lý bitstream và không chạy prover trong vòng lặp camera. Khi tích hợp, cần giữ đúng payload contract giữa bước đóng gói, embed và blind extract; native live channel hiện không tự gọi Circom/snarkjs verifier.

Table 1: Thành phần và hợp đồng dữ liệu chính.

Khối	Đầu vào	Đầu ra / trách nhiệm
Circom/snarkjs	m, s , circuit artifacts	Witness, Groth16 proof và public signals.
Payload packer	Message và proof bytes	Blob ứng dụng: 4 byte độ dài message, message, proof compact.
Annex-B/NAL reader	Byte stream H.264	NAL unit, header, EBSP/RBSP; đọc cả khi start code vắt qua chunk.
CAVLC parser/candidate collector	IDR I-slice Baseline/CAVLC	Cấu trúc residual và offset của sign-bit candidate.
Scheduler/patcher	Candidate, bit payload, khóa	Sửa sign bit, dựng lại NAL/Annex-B và xuất stego stream.
Blind extractor/verifier	Stego, khóa; proof và public inputs	Payload bytes; quyết định proof đúng/sai cho relation trong circuit.

Các hàng trên mô tả luồng ứng dụng ZK và các thành phần video liên quan; không nên hiểu rằng mọi API mode đang nói đầy đủ tất cả khối. Phần API bên dưới chỉ rõ hai đường HTTP job và WebSocket live, cùng giới hạn hiện tại của quan hệ giữa native stream với Groth16.

2 Benchmark và kết quả chính

Phần này được đưa lên trước phần giải thích mã hóa để người đọc biết ngay điều gì đã chạy được. Số liệu bên dưới lấy từ run mới 20260924T150707Z_2be60c; chạy trên Windows 11 Pro, ASUS TUF Gaming F15 FX507ZM, Intel Core i7-12700H, RAM 25.37 GB, Python 3.12.10. Native pipeline đo CPU; GPU RTX 3060 không tham gia xử lý.

2.1 Thiết kế phép đo

Tập thử gồm 9 clip Y4M (Akiyo, City, Coastguard, Container, Deadline, Football, Foreman, Foreman 300-frame, Hall Monitor), mỗi clip ở 352×288 , 640×480 và 1280×960 : 27 cấu hình, tối đa 30 frame/cấu hình. Encoder là FFmpeg/libx264 Constrained Baseline, CAVLC, QP 22, GOP=1/all-intra, không B-frame, một thread và một slice/IDR. Hai cỡ lớn được phóng từ nguồn CIF nên là stress test phân giải, không phải nội dung camera native HD.

Table 2: Các số đo đại diện của file pipeline. Median được tính trong từng nhóm 9 clip ở cùng độ phân giải.

Chỉ số	352×288	640×480	1280×960
Median output FPS	215.23	66.96	12.95
Median native RSS (MB)	9.70	17.11	45.82
Median source-to-stego PSNR-Y (dB)	43.77	45.17	46.81
Số cấu hình qua gate	9/9	9/9	9/9

Output FPS bằng số frame encode chia thời gian embed native; nó không bao gồm capture camera, khởi động encoder hay thời gian scan capacity. RSS là peak lấy mẫu của tiến trình native embed, không phải toàn bộ ứng dụng và có thể bỏ lỡ spike ngắn. Trung vị giảm từ 215.23 xuống 12.95 FPS khi tăng độ phân giải 352×288 lên 1280×960 ; do đó kết quả không hỗ trợ kết luận đạt realtime 30 FPS ở mọi độ phân giải.

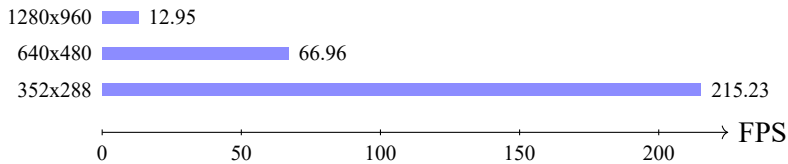


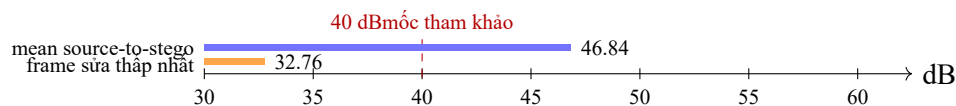
Figure 2: Median output throughput theo độ phân giải trong 27 case file pipeline. Thang ngang có mốc đến 200 FPS để thể hiện rõ các nhóm thấp hơn; cột 352×288 vượt mốc 200 và số 215.23 được ghi cạnh cột.

Đây là phép đo embed offline trên đoạn H.264 đã encode, không phải đo end-to-end từ cảm biến. Ở 640×480 , median 66.96 FPS cao hơn mục tiêu 30 trên clip ngắn của benchmark; ở 1280×960 , median 12.95 FPS thấp hơn mục tiêu. Kết luận hợp lý là throughput phụ thuộc mạnh vào độ phân giải và nội dung, chưa thể tuyên bố realtime tổng thể.

Table 3: Tóm tắt chất lượng ảnh và kết quả kiểm tra chức năng.

Đại lượng	Kết quả	Cách đọc
Gate chức năng	27/27	Embed, strict FFmpeg decode, blind extract đúng khóa và từ chối khóa sai.
Trung bình PSNR-Y source-to-stego	46.84 dB	Bao gồm cả nén H.264 lossy và nhúng.
Trung bình SSIM source-to-stego	0.99025	Tương đồng cấu trúc trung bình trên 27 cấu hình.
PSNR-Y thấp nhất trên frame bị sửa	32.76 dB	Một số frame có suy giảm thấy được.
Frame/case có PSNR dưới 40 dB	13/27	40 dB là mốc tham khảo, không phải gate chức năng.
Capacity scan	median 9.61 s; max 96.25 s	Tách biệt với embed time; tìm candidate tốn thời gian đáng kể.

PSNR-Y đo sai khác cường độ luma theo dB; cao hơn thường đồng nghĩa sai số bình phương thấp hơn. SSIM nằm gần 1 khi cấu trúc hai ảnh tương tự. Source-to-stego gộp hai nguồn sai khác (nén và nhúng), còn cover-to-stego đối chiếu bản H.264 sạch với bản stego để cô lập ảnh hưởng nhúng. Mỗi case chỉ mang payload vào một phần nhỏ frame, nên PSNR trung bình cả clip có thể che mất frame lỗi; cần xem giá trị theo từng frame trong CSV.

**Figure 3:** So sánh PSNR-Y trung bình toàn cấu hình với frame bị nhúng có chất lượng thấp nhất.

Khoảng cách giữa 46.84 dB toàn cấu hình và 32.76 dB frame xấu nhất cho thấy vì sao chỉ nhìn trung bình là chưa đủ. Hệ thống qua gate giải mã/khôi phục nhưng chưa đảm bảo ngưỡng chất lượng đồng đều; cải thiện candidate selection và giảm biến dạng trên từng frame là việc ưu tiên.

2.2 Proof system và luồng camera

Table 4: Đo thật Groth16 và PLONK trên cùng circuit, witness và message; mỗi hệ có một trial.

Chỉ số	Groth16 BN254	PLONK KZG	Diễn giải
Witness	433.93 ms	422.59 ms	Có thời gian khởi động Node.
Prove	2.581 s	831.695 s	PLONK khoảng 13.86 phút ở phép đo này.
Verify	848.36 ms	1,013.60 ms	Một trial mỗi hệ, chưa có khoảng tin cậy.
Peak RSS prover	1.12 GB	19.82 GB	Lấy mẫu mỗi 10 ms, có thể hụt peak ngắn.
Proof serialized	129 B compact / 808 B JSON	2,251 B JSON	Compact serializer hiện có cho Groth16.
Setup/artifact	zkey có sẵn; provenance chưa audit	setup 668 s; proving key 21.88 GB	PLONK dùng transcript PoT local power 18.

Trong phép đo đơn này, Groth16 có proof compact nhỏ hơn và prove nhanh hơn đáng kể. Đây không phải so sánh ổn định theo thống kê và không bao gồm Halo2, STARK/RISC Zero hay Bulletproofs vì chưa có

triển khai cùng circuit trong repo. Circuit có 62,553 constraints, chứng minh quan hệ hash/commitment của message và secret; nó không tự chứng minh nguồn camera hoặc toàn vẹn toàn bộ video.

Table 5: Một lần chạy camera E2E 60 giây có payload chứa proof Groth16 thật (run 24).

Đại lượng	Kết quả	Cách đọc
Frame ghi/giải mã	450 / 450	FFmpeg strict decode thành công.
Throughput	7.608 FPS active; 7.331 gồm startup	Thấp hơn mục tiêu 30 FPS.
Proof và payload	Proof 129 B; verify đúng	Proof tạo trước capture, verify sau extraction.
Negative checks	Khóa sai và message sửa bị từ chối	Xác nhận đúng payload/statement trong lần chạy.
PSNR-YUV420 toàn video / frame sửa thấp nhất	65.23 / 46.78 dB	Mean SSIM luma 0.999928.
Process-tree peak RSS	188.84 MB	Gồm tiến trình API, FFmpeg và native; lấy mẫu mỗi 100 ms.

Run 24 chứng minh đường E2E camera đã truyền proof thật qua WebSocket/native H.264, blind-extract và verify sau capture. Proof-generation mất 4.016 s ngoài capture path; verification mất 2.074 s sau extraction. Đây là một run đơn: nó chứng minh hoạt động ở cấu hình đã thử, không chứng minh ổn định lâu dài hay realtime 30 FPS.

3 Từ ảnh đến H.264: nhìn thấy từng bước

3.1 RGB, Y, Cb, Cr và lấy mẫu màu

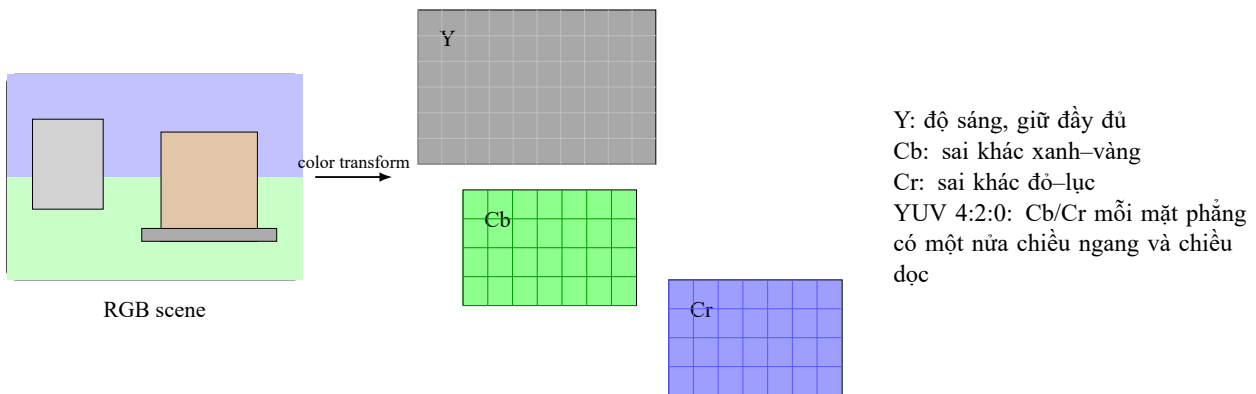
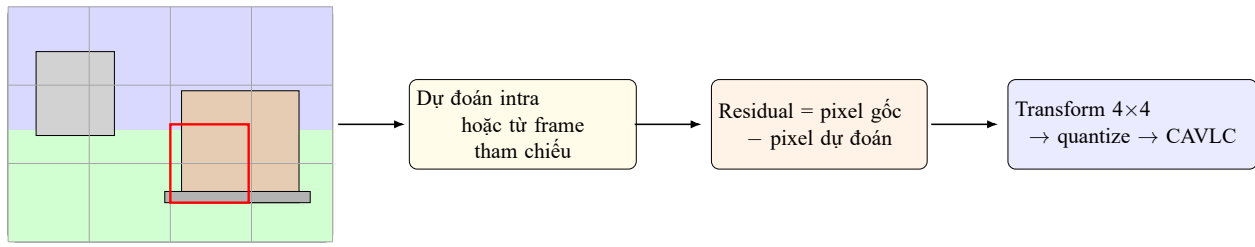


Figure 4: Minh họa tự dựng RGB được phân thành ba mặt phẳng. Độ phân giải mỗi mặt phẳng màu thực tế phụ thuộc pixel format; benchmark dùng YUV420.

RGB mô tả màu theo ba thành phần đỏ, lục, lam; encoder thường chuyển sang biểu diễn luma/chroma. Với YUV420, mỗi mẫu Cb/Cr đại diện vùng 2×2 pixel, vì mắt nhạy với độ sáng hơn sai khác màu. Frame 352×288 do đó giữ mặt phẳng Y 352×288 , còn mỗi chroma plane là 176×144 . Hình này là sơ đồ giải thích, không phải frame benchmark.

3.2 Macroblock, dự đoán và residual



Frame chia thành macroblock 16×16

Figure 5: Macroblock là vùng ảnh được mã hóa cùng thông tin dự đoán và residual; ô đỏ là ví dụ vùng được phóng to.

Encoder không chỉ lấy các pixel rồi biến thẳng thành bit. Nó hình thành dự đoán, trừ dự đoán khỏi block nguồn để lấy residual, biến đổi residual thành hệ số tần số, lượng tử hóa các hệ số và cuối cùng entropy-code chúng. Với all-intra, frame không tham chiếu frame trước; trong mã native hiện tại đường embed tập trung IDR I-slice/CAVLC.

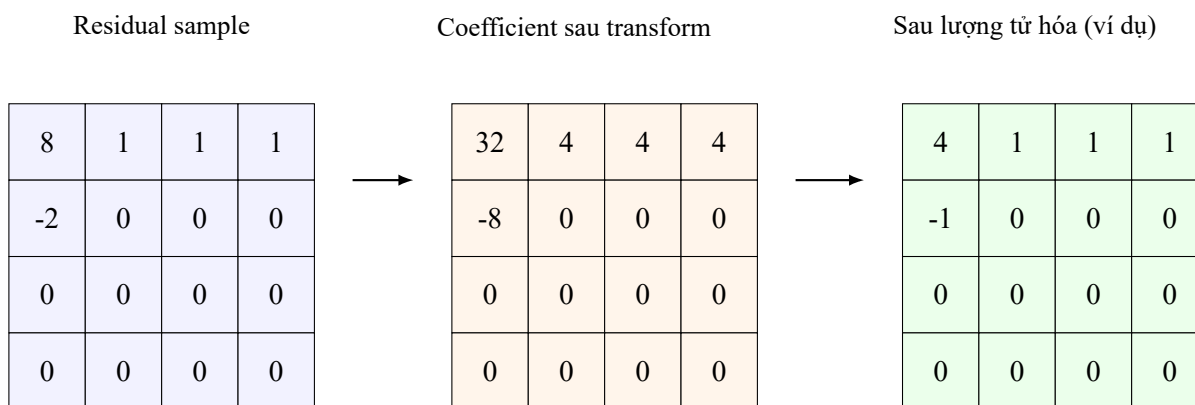
3.3 Lượng tử hóa và bảng QP

H.264 không bắt buộc một ma trận quantization duy nhất áp dụng cho mọi block. QP xác định mức bước lượng tử hóa; scaling lists (nếu có) điều chỉnh tương quan tần số và loại block. Dưới đây là chu kỳ hệ số bước cơ sở chuẩn hóa theo QP, với $q = \lfloor QP/6 \rfloor$, $r = QP \bmod 6$, và $Q_{step} = V_r 2^q$; hệ số V_r theo quan hệ QP trong ITU-T H.264 phiên bản 16. Đây là mô hình bước lượng tử để đọc QP, không phải thay thế phép biến đổi/rounding integer chính xác ở encoder.

Table 6: Hệ số bước lượng tử cơ sở theo phần dư QP modulo 6.

$r = QP \bmod 6$	V_r	QP mẫu	Q_{step} chuẩn hóa
0	0.6250	0	0.6250
1	0.6875	1	0.6875
2	0.8125	2	0.8125
3	0.8750	3	0.8750
4	1.0000	4	1.0000
5	1.1250	5	1.1250

Khi QP tăng 6, bước cơ sở tăng gấp đôi; hệ số nhỏ dễ bị lượng tử về 0. Benchmark đặt QP=22, tương ứng $q = 3$, $r = 4$; bước cơ sở chuẩn hóa bằng $1 \times 2^3 = 8$ trước khi tính đến scaling/shift của từng vị trí. QP cao tiết kiệm bit hơn nhưng thường làm giảm chi tiết. Hệ số sửa để nhúng được chọn sau lượng tử hóa, vì thay đổi coefficient trước bước này sẽ bị encoder ghi đè.



Số liệu được dựng để minh họa phép chia và làm tròn, không phải trace từ file encode. Trong benchmark H.264, bộ mã hóa áp dụng integer transform, scaling factor, shift và rounding theo vị trí; vì vậy không dùng phép chia 8 giản lược để tái tạo bit-exact.

Figure 6: Ví dụ số học minh họa: một residual block tạo ra hệ số biến đổi, sau đó lượng tử hóa làm hệ số nhỏ trở thành 0.

Ví dụ cho thấy lượng tử hóa làm thừa ma trận coefficient để entropy coder mã hóa ít symbol hơn, đổi lại mất thông tin không thể khôi phục hoàn toàn. Hệ thống nhúng sửa dấu coefficient đã lượng tử hóa và không thay QP; vì vậy nó không thể khôi phục chi tiết đã mất từ bước nén.

3.4 Cấu hình encode tái lập

Benchmark gọi Ffmpeg/libx264 trên Y4M. Phần chính của lệnh benchmark như sau:

```
ffmpeg -i input.y4m -frames:v 30 \
-vf scale=W:H:flags=lanczos,format=yuv420p \
-c:v libx264 -preset ultrafast -tune zerolatency \
-profile:v baseline -qp 22 -g 1 -bf 0 -threads 1 -pix_fmt yuv420p \
-x264-params "keyint=1:min-keyint=1:scenecut=0:repeat-headers=1:slices=1:threads=1" \
-f h264 output.h264
```

Các flag quyết định điều kiện benchmark, không phải quy tắc của mọi H.264. Constrained Baseline dùng CAVLC; -g 1 -bf 0 tạo all-intra không B-frame; một thread/slice làm run kiểm soát dễ lập hơn nhưng khác cấu hình camera thực tế.

4 NAL, slice và loại frame I/P/B

4.1 Từ Annex-B bytes đến NAL header

Annex-B là cách đóng khung byte stream H.264 bằng start code 3 hoặc 4 byte; NAL parser là phần mềm tìm marker, tách NAL, đọc header và chuyển EBSP/RBSP. Đầu stream ví dụ: 00 00 00 01 67 ... (SPS), 00 00 00 01 68 ... (PPS), 00 00 00 01 65 ... (IDR slice). Byte header có forbidden_zero_bit, nal_ref_idc, nal_unit_type; giá trị type 5 nhận diện IDR slice. Start code không phải là header NAL.

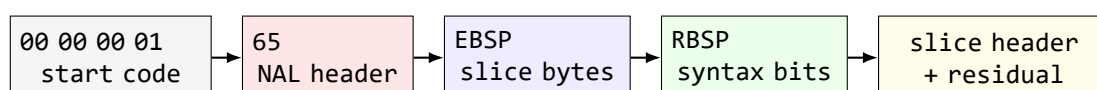


Figure 7: Giải phẫu NAL IDR minh họa. 0x65 có NAL type 5; payload sau header cần unescape trước khi đọc syntax.

Parser tìm start code qua luồng byte, lấy NAL header, rồi bỏ byte emulation-prevention 03 để khôi phục Rbsp. Trong streaming, start code có thể bị chia giữa hai chunk; reader giữ đoạn đuôi chưa đủ marker trước khi đọc chunk tiếp theo. Vì Annex-B là framing, parser mới là thuật toán xử lý cấu trúc bên trong từng unit.

4.2 I/P/B không đồng nhất với NAL type

I, P, B là kiểu slice/dự đoán; IDR là điểm làm mới trạng thái tham chiếu được báo bởi loại NAL. Slice header có trường `slice_type`; mã hóa chuẩn modulo 5 cho biết P/B/I/SP/SI. Ví dụ `slice_type=2` là I, còn `slice_type=7` cũng tương đương I (giá trị lặp cho phép). Một I-slice không nhất thiết là IDR; P/B-slice thường nằm trong NAL type 1. B-frame có thể dự đoán hai chiều nhưng không phải cứ có B là luôn dùng hai tham chiếu trong mọi macroblock.

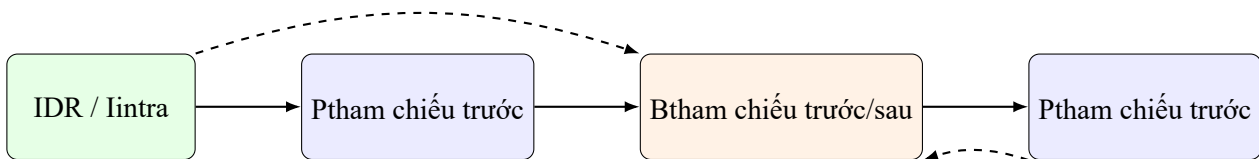


Figure 8: Ví dụ quan hệ tham chiếu theo thứ tự hiển thị, chỉ minh họa khái niệm. Benchmark của dự án dùng GOP=1 nên không có chuỗi P/B này.

4.3 Hàm parser trong source

```
# src/bitstream/h264.py
NALParser.parse(data) -> List[NALUnit]
NALParser._find_start_codes() / _extract_nal_unit(...)
iter_annex_b_nal_units(path, chunk_size=64*1024) -> Iterator[NALUnit]
SliceHeaderParser.parse() -> SliceHeader
MacroblockParser.parse_macroblock() -> MacroblockData
TraceableCAVLCParser.extract_with_offsets(nal, sps, pps, ...) -> dict
```

Parser NAL tìm start code, tách unit, đọc ba trường header và bỏ emulation-prevention bytes. Streaming iterator giữ dữ liệu qua ranh giới chunk. Slice-header parser đọc Exp-Golomb fields như first macroblock, slice type và PPS id, rồi các trường phụ thuộc SPS/PPS. Hàm macroblock parser đọc loại macroblock; với P-slice inter macroblock, đọc reference index và motion-vector difference; với intra macroblock, đọc prediction mode; sau đó xử lý coded-block-pattern, QP delta và residual. Traceable CAVLC parser giải mã coefficient đồng thời ghi offset bit để patch đúng vị trí. Các hàm cùng tên ở listing trên là tên trong source Python; native embedding benchmark hiện chỉ dùng IDR I-slice Baseline/CAVLC.

Đường native hẹp hơn parser Python tổng quát. Nó tách Annex-B, ghi nhớ SPS/PPS, lọc và giải mã IDR I-slice, rồi thu offset đầu của trailing-one. Các tên hàm của từng bước được liệt kê trong mã bên dưới. Native embedding hiện nhắm Baseline/CAVLC IDR I-slice; không suy luận rằng đường này đã sửa được mọi P/B/GOP.

```
// native/src/cavlc_stream.cpp (ược àphn ếmim tra ỗili)
split_annex_b(annex_b)
inspect_baseline_idr_headers(annex_b)
decode_baseline_i_idr_slices(annex_b)
collect_cavlc_trailing_one_sign_candidates(slices)
patch_annex_b_nal_rbsp(annex_b, nal_index, patches)
auto units = split_annex_b(annex_b);
for (const auto& nal : units) {
    const auto rbsp = nal.rbsp();
    if (nal.nal_unit_type == 7) parse_baseline_sps(rbsp);
    else if (nal.nal_unit_type == 8) parse_baseline_pps(rbsp);
    else if (nal.is_idr()) decode_baseline_i_idr_slice(rbsp, sps, pps);
}
```

Đoạn mã là sơ đồ rút gọn theo trình tự thật: giữ SPS/PPS để diễn giải slice, chỉ đi vào decoder với IDR. Hàm thực tế còn kiểm tra PPS/SPS có tồn tại và parse header trước khi bắt đầu đọc macroblock residual; vì vậy parse thất bại thì không nên coi bitstream là input được hỗ trợ.

5 Residual 4×4 , CAVLC và vị trí nhúng

Một residual block 4×4 sau transform/quantization là ma trận coefficient có thể thưa. CAVLC quét hệ số theo thứ tự zig-zag và mã hóa token gồm số hệ số khác 0 (TotalCoeff), số trailing-one (TrailingOnes), dấu, level và run/zero. Candidate của hệ thống là sign bit gắn với trailing-one có magnitude 1; nhúng bit 0/1 bằng cách chọn dấu âm/dương, không thay magnitude. Ma trận dưới đây là minh họa cấu trúc, không phải dump coefficient benchmark.

Coefficient trước					Sau nhúng bit			
4	1	0	0	→	4	-1	0	0
0	-1	0	0		0	-1	0	0
0	0	0	0		0	0	0	0
0	0	0	0		0	0	0	0

Bit payload đổi dấu $+1 \leftrightarrow -1$; magnitude giữ nguyên. Đây là ví dụ sign-bit, không phải toàn bộ codeword CAVLC.

Figure 9: Minh họa một sign bit trong ma trận coefficient thưa. Bộ giải mã CAVLC cần giữ chính xác cấu trúc token và offset bit.

```
// native/src/cavlc_stream.cpp (tên hàm ʊthc trong ʊd án)
decode_cavlc_luma_block(...)
decode_baseline_i_idr_slice(...)
collect_cavlc_trailing_one_sign_candidates(slices)
patch_annex_b_nal_rbsp(annex_b, nal_index, patches)
```

Candidate collector chỉ thêm vị trí khi token có sign-bit offset; mỗi candidate ghi NAL index, macroblock, loại residual, block index và bit offset. Patcher chuyển EBP sang RBP, áp dụng patch có độ dài cố định, dừng lại EBP và assemble Annex-B. Giữ độ dài patch cố định quan trọng vì thay đổi codeword VLC có thể làm lệch parser của mọi block sau. Gate strict decode xác nhận decoder chấp nhận stream; nó không thay phép đo PSNR/SSIM.

Trong pipeline Python ZK, payload được pack theo trường độ dài 4 byte big-endian, message, rồi proof 129 byte. Message 12 byte cho blob 145 byte, tức cần $145 \times 8 = 1,160$ bit candidate nếu không có framing khác. Native live API hiện dùng hợp đồng payload/channel riêng; code API không tự chạy prover/verifier cùng lúc. Sự tách biệt này phải được tính đến khi đánh giá end-to-end.

6 Groth16: payload, circuit và setup

6.1 Quan hệ được chứng minh

Circuit nhận message m và secret s ; tính $h = \text{SHA256}(m)$ và commitment $c = \text{SHA256}(h \parallel s)$, đồng thời kiểm tra độ dài message. Public signals là h , c và length; secret là private witness. Benchmark đo 62,553 constraints. Verifier trả true chỉ khi proof hợp lệ với public signals/verification key đã dùng; sửa public input làm lần thứ bị bác bỏ.

Proof không chứng minh rằng camera cụ thể đã quay video, timestamp là thật, hay toàn bộ stream không bị chỉnh sửa. Để đạt các tính chất đó, circuit hoặc một cơ chế ký/attestation khác phải ràng buộc commitment với dữ liệu video, thiết bị và thời gian; đây chưa phải kết quả hiện tại.

6.2 Các hàm và artifacts

Python `embed(...)` trong `src/embedder.py` gọi bridge để tạo proof, serialize proof 129 byte, pack payload, phân tích cover, chọn candidate an toàn, nhúng bit rồi gọi reconstructor dựng stream. Phía trích xuất khôi phục payload bytes trước khi client giải mã proof và gọi Groth16 verifier. Circuit compilation tạo R1CS/WASM/witness generator; trusted setup tạo proving key và verification key, không chạy lại theo từng video.

```
# src/embedder.py - òlùng ệnhíp ựv chính (rút ọgn, chia qua các helper)
proof, public = bridge.generate_proof_for_payload(message, secret_key)
proof_bytes = bridge.proof_to_bytes(proof)
payload_blob = pack(message, proof_bytes)
coeffs, frame_data, nC, lengths, overrides, safe = _resolve_video_analysis(...)
embedder = PayloadEmbedder(max_modifications_per_block=1)
modified, bits = embedder.embed_payload(coeffs, payload_blob,
                                       nC_map=nC, nal_length_map=lengths,
                                       t1_override_map=overrides,
                                       frame_verified_data=frame_data, ...)
BitstreamReconstructor().reconstruct_video(video_path, modified, output_path, ...)
```

Tên helper trong đoạn rút gọn diễn tả bước xử lý. Trong source, prover bridge, bộ nhúng payload và bitstream reconstructor chịu trách nhiệm các bước tương ứng. Kết quả nhúng được lọc theo block mà reconstructor thật sự áp dụng; danh sách candidate dự kiến không tự động được coi là bit đã ghi.

7 API và kết nối

7.1 HTTP file jobs

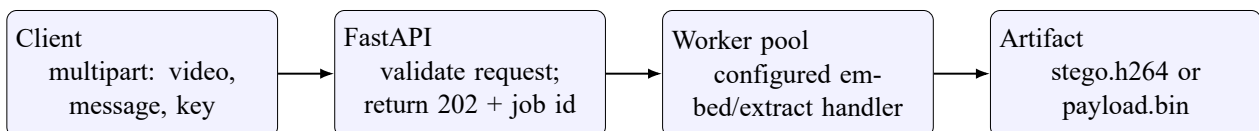


Figure 10: HTTP file flow. Client upload không phải kết nối camera trực tiếp; job chạy trong worker và artifact được tải khi hoàn tất.

Embed gửi multipart video cùng message và key tới `/api/v1/jobs/embed`; service trả HTTP 202 với job id. Sau đó client poll endpoint trạng thái job rồi tải artifact qua endpoint artifact. Extract gửi stego video, key và giới hạn độ dài; artifact trả payload. `/health` kiểm tra service. HTTP route verify nhận stego/original/key ở mode tương ứng; native service hiện được khởi tạo với verify handler tắt, nên không giả định endpoint đó khả dụng trên mọi cấu hình.

7.2 WebSocket camera stream

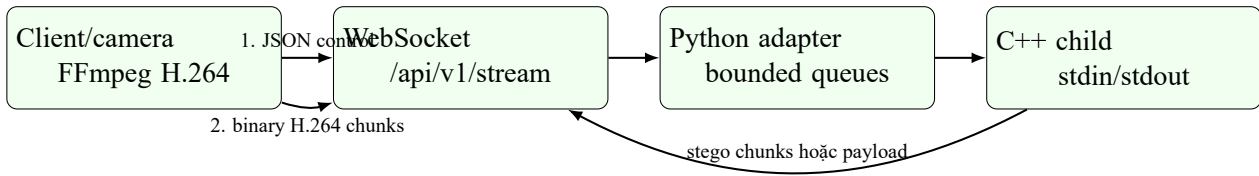


Figure 11: WebSocket stream là giao thức điều khiển + media chunks; output embed là binary stream, còn extract trả payload qua JSON.

Client mở WebSocket có Bearer authorization rồi gửi JSON control: thao tác, key, và khi embed thì message; extract gửi giới hạn độ dài payload. Sau phản hồi ready, client gửi từng H.264 chunk dạng binary; adapter chuyển sang stdin tiến trình C++ theo backpressure. Khi embed, output native được gửi lại dưới dạng binary chunks và kết thúc bằng thông điệp trạng thái; khi extract, payload được trả JSON Base64. Code giới hạn buffer 64 KiB high-water/16 KiB low-water và ghi nhận native resource samples.

```

# src/api/native_handlers.py - quy trình stream (uỷlc)
start = json.loads(control_text)
process = await asyncio.create_subprocess_exec(
    executable, *arguments, stdin=PIPE, stdout=PIPE, stderr=PIPE)
await websocket.send_json({"type": "ready", "operation": operation})
# binary websocket chunks -> native stdin; native stdout -> websocket
await process.stdin.drain() # backpressure
await websocket.send_bytes(chunk) # embed output
  
```

Luồng này thể hiện dữ liệu nào đi qua từng hop, nhưng không tạo ra TLS, camera attestation hay proof verification tự động. API key/payload control và media stream là dữ liệu đầu vào; client vẫn cần nhận kết quả, khôi phục payload và kiểm tra Groth16 theo đúng circuit contract nếu đang dùng luồng ZK.

8 Kết luận và giới hạn

Kết quả xác nhận file pipeline chạy thành công ở 27/27 cấu hình thử, có strict decode và blind extraction đúng/sai khóa; run camera E2E cũng mang proof Groth16 thật qua stream, trích xuất và xác minh sau capture. Camera throughput là 7.608 FPS, thấp hơn 30 FPS; chất lượng frame xấu nhất trong file benchmark là 32.76 dB; năng lực native được xác nhận chủ yếu ở IDR I-slice CAVLC all-intra. Chưa có cơ sở khẳng định hỗ trợ mọi GOP, P/B-frame, CABAC hoặc đạt realtime ở mọi kích thước.

Ưu tiên kỹ thuật tiếp theo là lặp lại camera E2E proof-bearing nhiều lần và tăng tốc capture-to-output, đồng thời áp ngưỡng chất lượng theo từng frame. So sánh với hệ khác cần dùng cùng video, QP, payload, độ phân giải và điều kiện host; phép đo hiện tại chưa chứng minh ưu thế tổng thể.

Artifacts benchmark được dùng

- Media run 20260924T150707Z_2be60c: video_pipeline_new.json.
- Per-frame CSV: quality_per_frame_new.csv trong cùng thư mục run.
- ZKP: zkp_new.json; camera: realtime_camera_runs_20260924.json.
- Quy trình chạy: NEW_BENCHMARKS.md.